Malaysia data center map — project spec

A plain-English guide for building a free, DIY map that shows where data centers are in Malaysia, and roughly how much power each one uses.

---

1. What this project is

This is a small website (or app) that:

- Shows a **map of Malaysia**
- Puts a **dot on the map for every known data center**
- Makes each dot **bigger or a warmer color** based on how much power that data center uses (in megawatts, shortened as **MW**)
- Lets you click a dot to see details: name, operator, address, estimated power, and where that info came from
- Lets **anyone report a mistake** they spot, so the map gets better over time instead of relying only on you

Think of it as your own free, Malaysia-only version of paid tools like Baxtel — just smaller in scope, community-checked, and built by you.

---

2. What "done" looks like (the goal)

A working v1 is done when you have:

- A list of **at least 30–50 real Malaysian data centers** with their locations
- Each one has a **power estimate**, even if it's a rough guess, labeled as "confirmed" or "estimated"
- A map you can open in a browser that shows all of them
- A **background job that checks for new information automatically**, instead of you searching by hand every time
- A **public repository** anyone can view, with a simple way for people to flag wrong or outdated info

---

3. Scope (what's in, what's out)

In scope:

- Data centers physically located in Malaysia (Peninsular + Sabah/Sarawak)
- Location, operator, power capacity (confirmed or estimated), status (operating / under construction / planned)
- A static map, refreshed on a schedule (not truly "live")
- An open-source codebase and dataset that other people can inspect, correct, and reuse

Out of scope (for v1 — you can add these later):

- Real-time temperature or live power readings (this data is private; nobody publishes this)
- Instant, minute-by-minute updates (data centers take years to build, so weekly or monthly checks are enough)
- Countries other than Malaysia

---

4. Key terms explained

You'll see these words a lot. Here's what they mean in plain English:

Term	What it means
MW (megawatt)	A unit of power. A small data center might use 5–10 MW. A huge one can use 500+ MW.
PUE (Power Usage Effectiveness)	A score showing how efficient a data center's cooling is. 1.0 is perfect (no waste). 2.0 means it uses double the power it really needs, just for cooling.
Hyperscale	A very large data center, usually built by a big company like Google, Microsoft, or Amazon.
Colocation	A data center where many different companies rent space, instead of one company owning the whole building.

Term	What it means
Overpass API	A free tool that lets you search OpenStreetMap's data by asking questions like "show me every data center in Malaysia."
OpenStreetMap (OSM)	A free, editable map of the world, built by volunteers — like Wikipedia, but for maps.
GeoJSON	A simple text file format for storing map locations (points, lines, areas) that most mapping tools understand.
API	A way for one piece of software to ask another piece of software for data, automatically.
RSS feed	A simple, standard "list of latest articles" that a news site publishes, meant to be read by software instead of a person.
Google Sheets API	A way for a script to open, read, or write to a Google Sheet automatically — no person needs to click into the browser. This is how the automation writes new findings into your spreadsheet.
Cron job / scheduled job	A task that runs automatically on a timer (e.g. "run this every Monday") instead of you clicking a button.
LLM (large language model)	An AI model (like Claude) that can read text and pull out specific facts from it, such as "what MW number is mentioned in this article."
Pending queue / review queue	A holding area for new information the automation found, which a human checks before it goes live — so mistakes don't get published automatically.
Open source	Making your code (and here, your data) publicly viewable, so anyone can inspect it, suggest fixes, or reuse it.

Term	What it means
Pull request	A way for someone else to propose a specific change to your project, which you can review and accept or reject.
Geocoding	Turning an address (like "1 Jalan Wan Kadir, KL") into map coordinates (latitude and longitude).
Latitude / longitude	The two numbers that pinpoint any exact spot on a map.

---

5. The data you will collect (data model)

For every data center, store these fields. This is the "shape" of your data — think of it as spreadsheet columns.

Field name	Example	Notes
`name`	"YTL Johor Data Center"	The facility's name
`operator`	"YTL Data Centers"	The company running it
`address`	"Kulai, Johor"	As specific as you can get
`latitude`	1.6608	From geocoding or satellite check
`longitude`	103.6046	From geocoding or satellite check
`capacity_mw`	72	The power number, in megawatts
`capacity_type`	"confirmed" or "estimated"	Be honest about which ones are guesses
`capacity_source`	"Company press release, Aug 2025"	Always write down where the number came from

Field name	Example	Notes
`status`	"operating" / "under construction" / "planned"	
`connection_voltage`	"132kV"	Optional — a clue to size when MW is unknown
`verification_status`	"confirmed" / "needs review" / "community flagged"	Tracks whether a human has checked this row
`last_updated`	"2026-08-26"	So you know how stale the entry is
`report_url`	link to a GitHub issue or form response	Where a correction, if any, was submitted

Tip: Start this as a simple spreadsheet (Google Sheets or Excel). You can convert it into other formats later.

---

6. Where the data comes from (sources)

Use these sources. Items 3, 4, and 6 get automated — see Section 8 for how.

1. **OpenStreetMap (via the Overpass API)** — free, and some Malaysian data centers are already tagged there as `telecom=data_center`. This gives you a free starting list with coordinates.
2. **Free listings on public data center directories** (e.g. datacentermap.com, datacenters.com) — these show name, operator, and rough address for free. Don't try to access anything sitting behind a paywall or login — that content belongs to the site and copying it would break their terms of use.
3. **Company press releases and news articles** — announcements mentioning "[company name] Malaysia data center MW." Many companies announce their own capacity publicly because it's good marketing for them. *(Automated — see Section 8.)*
4. **Government sources** — MIDA (Malaysian Investment Development Authority) and TNB (Tenaga Nasional Berhad) occasionally name specific projects and sizes in their announcements. *(Automated — see Section 8.)*
5. **Satellite imagery (Google Earth, free)** — used only to confirm a location is really a data center, or to estimate size when no MW figure is public. Look for: large air conditioning/generator clusters, a fenced compound, very little parking, and closeness to an electrical substation.

6. **Bank Negara Malaysia's national reports** — good for double-checking your total against the country-wide figure (Malaysia's total live capacity was reported around 784 MW as of late 2025). *(Automated — see Section 8.)*

---

7. How to collect the data (workflow)

1. **Pull the free OSM list first.** Use the Overpass API (there's a web tool called Overpass Turbo that needs no coding) to search for `telecom=data_center` inside Malaysia's borders. Save the results into your spreadsheet.
2. **Add anything OSM missed.** Browse the free listings on the public directory sites and add any facility not already in your list.
3. **Let the automated pipeline do the searching for you** (Section 8) instead of manually Googling each company. It will drop new candidate facts into a review queue.
4. **You just approve or reject** what the pipeline finds — a quick check, not a research session.
5. **Estimate what's left.** For entries with no public number at all, use the connection voltage as a rough size clue, or estimate from the building's footprint size on satellite maps. Mark these as "estimated."
6. **Clean up duplicates.** The same data center sometimes appears under slightly different names across sources — merge these into one row.
7. **Sanity-check the total.** The automated Bank Negara check (Section 8) does this for you automatically each quarter.

---

8. Automating the search-heavy sources (this replaces manual checking)

Manually re-Googling every company, checking MIDA and TNB's news pages, and re-reading Bank Negara's quarterly reports doesn't scale — you'd be doing it forever. Instead, build a small **four-stage pipeline** that runs on a timer and only asks for your attention when it actually finds something new.

Where all of this actually lives: one Google Sheet, two tabs.

This is the part that's easy to leave vague, so to be explicit: there is only **one spreadsheet file** for this whole project. It just has two tabs (like two pages in the same notebook):

- A **"Pending" tab** — where new, unverified findings land.

- A **"Main" tab** — your trusted, confirmed data. This is the *only* tab that ever gets exported to the map.

The automated pipeline below writes into the **Pending** tab using something called the **Google Sheets API** — a way for a script to open and edit a spreadsheet automatically, without a person clicking into it. Nothing the automation finds ever touches the Main tab directly. You review the Pending tab periodically and move (or copy) approved rows into Main yourself. Only Main ever gets turned into the GeoJSON file the map reads — so nothing unverified ever reaches your public map.

The four stages:

1. **Watch** — Set up free, no-signup **Google News RSS feeds** for search terms like `data center Malaysia MW`, `MIDA data centre`, and `TNB data centre`. An RSS feed is just a standing search that updates itself — you don't need to re-search it by hand. Also check whether MIDA and TNB publish an RSS feed on their newsroom pages; if not, a simple script can save a copy of the page and compare it to last time, flagging anything new.
2. **Fetch** — On a schedule (e.g. once a week), a small script downloads the full text of any new articles found in step 1. For Bank Negara, this step instead means downloading their newest Quarterly Bulletin PDF once it's published (roughly every 3 months).
3. **Extract** — Feed each article's (or report's) text into an AI text model — Claude's API is a good fit here — with instructions like: *"Read this article. If it mentions a Malaysian data center's name, operator, location, or power capacity in megawatts, return that as structured data. If not, return nothing."* This turns messy news text into clean rows matching your data model from Section 5.
4. **Queue for review** — The script writes each finding as a new row in the **Pending tab** of your Google Sheet, using the Google Sheets API. It never writes to Main, and it never touches the map. You then spend a few minutes confirming or rejecting each Pending row — a quick check, not a research task.

Real starting sources for the pipeline to watch (verified, not hypothetical):

Source	What's there	How to fetch it	Okay to scrape?
Equinix's facility pages (e.g. `equinix.com/data-centers/.../kuala-lumpur-data-centers`)	Each facility has a free downloadable PDF "technical spec sheet" with real power details (generator size,	Direct PDF download, no login	Yes — these are published for public download

Source	What's there	How to fetch it	Okay to scrape?
	power density per cabinet)		
MIDA media release archive (`mida.gov.my/media-release/`)	Government announcements, sometimes stating exact MW figures in the article text	Plain public webpage	Yes — public government press releases
TNB press/newsroom archive (`tnb.com.my/announcements/`, plus their newsclip PDF archive)	Named operators with exact MW figures, since TNB reports on its own data-centre electricity deals	Direct PDF and webpage access	Yes — TNB's own public relations material
Google News RSS (free feed, e.g. searching `data center Malaysia MW`)	Ongoing new articles as they're published	A standing RSS feed URL, no account needed	Yes — RSS feeds are built for exactly this

A general rule for deciding if a source is fair game: if the page or file exists so people can freely read or download it (a press release, a spec sheet, a news archive), it's fine. If it sits behind a login or a payment (like Baxtel's paid database), leave it alone — that's the one line not to cross.

How to run this on a schedule for free:

- Since your code will already live on GitHub (Section 10), use **GitHub Actions** — it has a free "scheduled workflow" feature that can run your fetch script automatically every week or month, with no server needed.

Bank Negara total-check, specifically:

- Once the extraction step pulls the newest national total MW figure from their latest report, have the script automatically add up your own spreadsheet's `capacity_mw` column and compare the two numbers. If they're very different, it prints a warning — this replaces you manually remembering to check.

---

9. Tech stack (the tools to build with)

Keep this as simple as possible — Malaysia's data center count is small enough that you don't need anything fancy.

Data storage

- **Start with:** a Google Sheet or CSV file. This is your source of truth while you're collecting data.
- **Once stable:** export it to a **GeoJSON** file (a simple format that map tools can read directly).

Why a Google Sheet, and not something else — the honest comparison:

The right storage tool depends on the size of the job. Here's why the bigger options don't earn their cost here:

Option	What it's actually built for	Why it's not the pick here
Google Sheets *(chosen)*	Small lists a human wants to browse, filter, and edit by eye	Fits: this project has roughly 50–200 rows and needs a person to approve or reject entries — a spreadsheet makes that a glance, not a query
SQLite (a single-file database)	An app that needs a private, structured database file, without running a separate server	Works technically, but you'd need an extra tool just to *look at* your data, and a database file doesn't show a clean history in GitHub the way a CSV does
A full DBMS (like Postgres or MySQL — a database server other apps connect to)	Many people or programs reading and writing the same data *at the same time*, at large scale	Overkill: this is a solo project with no simultaneous-users problem to solve, and running a database server costs money and upkeep for no real benefit at this size
A vector database (built for "search by meaning" using	Matching things by *similarity in meaning*, not exact values	Wrong tool: this project needs exact filters like "show me

Option	What it's actually built for	Why it's not the pick here
AI, e.g. "find data centers similar to this one")		sites over 50 MW in Johor," not fuzzy meaning-based search — there's nothing here for a vector database to do

If the project grows later — say, you expand beyond Malaysia, or want several people editing at once — a good upgrade path is **Supabase**, a free-tier hosted Postgres database that also gives you a built-in spreadsheet-style table editor. You'd get real database power without losing the "just look at the table" simplicity that makes Sheets easy to use now. The GeoJSON export step already keeps your storage separate from your map, so switching later won't require touching the frontend at all.

Map / frontend

- **Leaflet.js** — a free, beginner-friendly JavaScript library for building interactive maps. Very well documented, lots of tutorials.
- **Map tiles (the visual map background):** free tiles from **OpenStreetMap**, or **CARTO's free tier**, both work well with Leaflet.
- Each data center becomes a **marker** on the map. Marker size or color = the MW value.

Hosting

- **GitHub Pages**, **Netlify**, or **Vercel** — all free for a small static site like this. You just upload your HTML/JS/GeoJSON files and get a live public link.

Automated data collection (new)

- **Google News RSS feeds** — free, no account needed, for watching news mentions.
- **GitHub Actions** — free scheduled jobs to run your fetch-and-extract script weekly or monthly.
- **Claude API (or another LLM API)** — used to read fetched article/report text and pull out structured facts (name, MW, location).
- **A "pending" review tab or auto-created GitHub issues** — where automated findings wait for your approval before going live.
- **Python** (`requests` for fetching pages, `pandas` for handling the spreadsheet, `feedparser` for reading RSS feeds) — the glue that connects these steps.

Community feedback (new)

- **GitHub Issues** — the simplest, free way for anyone to report a wrong or outdated entry.

- Optional: a **free Google Form** linked from each map popup, for people who don't have a GitHub account, feeding into a "reported corrections" spreadsheet tab.

Summary table

Purpose	**Tool**	**Cost**
Store the data	Google Sheets → GeoJSON	Free
Draw the map	Leaflet.js	Free
Map background	OpenStreetMap / CARTO tiles	Free
Host the website	GitHub Pages / Netlify	Free
Watch for news	Google News RSS	Free
Run checks on a schedule	GitHub Actions	Free
Pull facts out of articles/reports	Claude API (or similar)	Pay-per-use, very low volume here
Hold automated findings for approval	Spreadsheet tab or GitHub Issues	Free
Collect public corrections	GitHub Issues / Google Form	Free

---

10. How the pieces fit together

Manual + map side:

Your spreadsheet (CSV)

|

▼

Convert to GeoJSON (a simple script or free online converter)

|

▼

Leaflet.js reads the GeoJSON file

Leaflet draws dots on an OpenStreetMap background

You upload the HTML + GeoJSON to GitHub Pages

Anyone can open the link and see your live map

Automated data collection side:

GitHub Actions runs your script on a schedule

Script checks Google News RSS + MIDA/TNB pages + BNM reports

New article or report text found?

Claude API reads the text, pulls out name / MW / location

Result added to a "pending review" list — NOT the live map yet

You glance at the pending list, approve or reject

Approved rows get added to your main spreadsheet

Community feedback side:

Someone spots a wrong entry on your public map

They click "Report an issue" (opens a pre-filled GitHub Issue or form)

You review the report

If correct, update the spreadsheet and mark the row "confirmed"

---

11. Making it open source

Yes — open-sourcing this is a good call. It means more eyes checking your data, and it's the same spirit as OpenStreetMap itself, which some of your data comes from.

- **Put the whole project on GitHub as a public repository** — both the code (map, scripts) and the data (spreadsheet/GeoJSON).
- **Add a license file:**
  - For your **code**, the **MIT license** is a simple, common, permissive choice.
  - For your **data**, check compatibility if you're including OpenStreetMap-derived data — OSM's own data is under the **Open Database License (ODbL)**, which requires giving credit and, in some cases, sharing alike. Read OSM's license summary before you publish, so your dataset stays compliant.
- **Write a short README** explaining what the project is, where the data comes from, how often it updates, and how someone can suggest a fix — this is what turns a personal project into something others can trust and contribute to.
- **Accept pull requests** — if someone technical wants to fix a row directly instead of just reporting it, GitHub lets them propose the exact change for your approval.

---

12. Community feedback & correction mechanism

Since a single person (you) can't verify everything forever, build in a lightweight way for others to flag problems.

Recommended setup:

- On each map marker's popup, add a **"Report an issue" link**. This can be a specially-built GitHub link that opens a new issue, pre-filled with that data center's name — so the reporter doesn't have to type much, just describe what's wrong.
- For non-technical visitors, also link a **free Google Form** ("See something wrong? Tell us here") as an easier alternative to GitHub.
- Route every report into the same **Pending tab** your automation writes to (Section 8) — so you have one single place, in one single spreadsheet, to process both AI-found updates and human-reported corrections.
- When you confirm a correction, update the `verification_status` field to "confirmed" and note the source, keeping the same honesty standard as the rest of your data.

This turns your project from "one person's spreadsheet" into a small, self-correcting community resource — without needing you to build a complicated system.

---

13. Build phases (suggested roadmap)

1. **Phase 1 — Research:** Collect 20–30 confirmed data centers into a spreadsheet using the sources above.
2. **Phase 2 — Static map:** Build a basic Leaflet map that plots those points from a GeoJSON file, with a popup showing name and MW on click.
3. **Phase 3 — Fill in the rest:** Add every remaining known facility, including estimated ones, clearly marked.
4. **Phase 4 — Automate the search:** Build the four-stage pipeline (watch, fetch, extract, queue) from Section 8, running weekly via GitHub Actions.
5. **Phase 5 — Go open source:** Publish the repo publicly with a license and README.
6. **Phase 6 — Add community feedback:** Add the "Report an issue" links and hook them into your review queue.
7. **Phase 7 — Polish:** Add a legend, color-code by status (operating / planned), and a search or filter box.

---

14. Rules to follow (keep this project safe and legal)

- Only use **publicly available, free information.** Never copy data sitting behind a paywall or login screen — that breaks the source's terms of use.
- Always **record where a number came from.** If you can't confirm a number, label it as an estimate — don't guess and present it as fact.
- Don't claim your map shows **real-time** anything. It shows your best research as of the last update date.
- Give proper **attribution to OpenStreetMap** wherever you display OSM-derived data — this is a condition of their free license, not optional.
- When automation pulls information from an article, **keep a human approval step** before it goes live — AI extraction can make mistakes, and this catches them before they reach the public map.
- When accepting public corrections, don't blindly trust every submission — verify against a real source before marking something "confirmed."

---

15. Keeping it updated

- Malaysia's data center industry moves slowly compared to how fast a website updates — new facilities take **1–3 years** to build.
- Let the automated pipeline (Section 8) do the weekly/monthly watching for you.

- Still do a **quick manual pass every 3–6 months** to catch anything the automation missed and to process any pending community reports.
- Keep the `last_updated` field honest for every row, so anyone using your map knows how fresh the data is.

---

16. Success checklist

- ☐ Spreadsheet with at least 30 real Malaysian data centers
- ☐ Every row has a location (latitude/longitude)
- ☐ Every row has a power estimate, labeled confirmed or estimated
- ☐ GeoJSON file exported from the spreadsheet
- ☐ Working Leaflet map showing all points
- ☐ Map hosted online with a shareable link
- ☐ Automated weekly/monthly fetch pipeline running via GitHub Actions
- ☐ New automated findings land in a review queue, not straight on the map
- ☐ Project published as a public, licensed GitHub repository
- ☐ A working "Report an issue" link on each map marker
- ☐ A note on the site explaining data sources and limitations